
GENERATING REALISTIC DOG IMAGES WITH A VARIATIONAL AUTOENCODER

A PREPRINT

8 December 2024

ABSTRACT

Generative models, particularly variational autoencoders (VAE), have shown to be able to capture highly complex data distributions and generate new samples from them. In this work, we implement a VAE to model the distribution of a set of dog images originating from real-world scenarios. We aim to use the VAE to generate new dog images. As a foundation, we first design and train a convolutional autoencoder (CAE) to efficiently reconstruct samples from the dataset. Building upon this foundation, we implement the VAE. Model performance is evaluated using reconstruction loss, Inception Score and Fréchet Inception Distance (FID), assessing reconstruction and image generation capabilities. Our final results show that while CAE excels in reconstruction, it cannot generate anything meaningful. The VAE successfully generates new samples with some structural fidelity, although challenges such as blurring and high FID scores highlight areas for improvement. The generated images do not look like realistic dogs, but provide significant improvement over the CAE.

1 Introduction

Generative models in computer vision have achieved impressive results with the introduction of variational autoencoders (VAEs) and generative adversarial networks (GANs) [7]. These models capture the data distribution itself, which is significantly more challenging than capturing conditional distributions, which is what discriminative models do. The results achieved by VAEs are impressive because they show that VAEs can model the distributions of very complex datasets, and can then generate new, unseen data from those distributions.

In this work, we implement a VAE to model the distribution of a complex, real-world dataset consisting of dog images. Our goal is for the decoder part of the VAE to generate realistic dog images given random samples from the latent space. First we implement a convolutional autoencoder (CAE) with a small architecture. Once the CAE can compress and decompress samples from the dataset successfully, we build the VAE using the CAE’s architecture as foundation. Thereby we keep the VAE as small and efficient as possible. This approach is advantageous as it is easier to focus on reconstruction first rather than generation, and successful reconstruction is the basis for successful generation. Moreover, comparing the CAE’s generative ability to the VAE’s generative ability gives a basis for evaluating the VAE.

We find that implementing a CAE that is capable of reconstructing images from the complex dataset is possible with a small architecture, but as expected, the CAE is not capable of generating anything meaningful. Making it a VAE significantly improves generation, however still far from being perfect, probably due to the small model capacity.

2 Related Work

Autoencoders (AEs) were first introduced in [8] as neural networks designed to compress input data into a simpler, meaningful representation and then reconstruct the original input from this representation. The space in which the compressed representations reside is called the latent space. The goal was for the compressed representation to retain important information about the original input, making it useful for tasks like clustering, dimensionality reduction, and denoising [1]. However, while AEs excel at reconstructing data, they are not particularly effective for generating new data. This limitation arises because their latent space encodes data in a deterministic way, which does not naturally support the generation of diverse, realistic samples.

Variational autoencoders (VAEs) were introduced by Kingma and Welling in [5] to address this limitation. VAEs extend AEs by making the latent space continuous and probabilistic rather than deterministic. This is achieved by having the encoder output two vectors: One representing the mean and the other the standard deviation of a Gaussian distribution. A sampling layer is then introduced to draw samples from this distribution [2]. This probabilistic approach enables VAEs to generate new, coherent samples by sampling from the learned latent space. While the outputs of VAEs may not always be as sharp as those generated by GANs, they often exhibit greater diversity and interpretability due to the structured latent space [9].

VAEs have been successfully applied in image generation, but also other tasks like image reconstruction [6, 12]. VAEs are commonly used to generate human faces [3, 11]. However, we cannot find the application of VAEs specifically to dog images in published articles.

3 Methods

3.1 Dataset

We use the Stanford Dogs Dataset [4], which contains 20580 images of dogs. The images are categorized into 120 different classes that resemble dog breeds and each image is annotated with a class label and a bounding box. There are no images containing dogs of multiple breeds. Every class has 148-252 associated images, so they are not perfectly balanced. We use the dataset without class labels and bounding boxes during model training. The goal is for the VAE to generate realistic dog images, without requiring it to generate specific dog breeds. All images are of 200×200 pixels or larger and are in RGB format with pixel values ranging from 0 to 255.

The images were taken in man-made environments, so there are humans in the background of many images and the backgrounds are extremely diverse and complex, making it difficult for the model to distinguish the dog from the rest of the image. In addition to that, generating realistic dogs requires the model to learn intricate and recognizable features of dogs, such as fur patterns and anatomy, which is a demanding task. Even without breed labels, the model still needs to account for the broad diversity in dog appearances across the dataset, as dogs from different breeds can look very different (high intra-class variability). The previous points make it extremely challenging for the model to approximate the underlying data distribution and thus to learn generating realistic dog images. This is one of the reasons why we choose this dataset. Another reason is that the dataset is trustworthy and reliable: The creators removed degenerate images and it is used in recent works to evaluate advanced generative models [10]. This is an advantage over alternatives. The original purpose of the dataset was to serve as challenging dataset in fine-grained image classification settings. But many of the challenges remain in image generation settings, as we describe above.

For model training, we split the dataset into training and validation sets, whereby 80% of images are assigned to the training set and 20% are assigned to the validation set. The assignment is done randomly. This gives us a training set size of 16464 samples.

3.2 Data Preprocessing and Augmentation

The data preprocessing pipeline begins by scaling pixel values to $[0, 1]$, ensuring efficient computation and stable training in combination with the ReLU activation function. Then the images are cropped to be of 64×64 pixels, to have consistent input sizes for the model. This size has shown to provide enough detail for the model to learn while being relatively small, allowing for a smaller network architecture and efficient computation.

Our task usually requires a large amount of data and since 16464 images may be too few for the model to learn and generalize well, we make up for the potential lack of data by augmenting it. The following transformations are applied every time a new batch is fetched during training: The cropping we mentioned earlier is done randomly, so the 64×64 pixels to which an image is cropped are chosen at random every time the image is fetched during training. Afterward, the images are flipped horizontally with a probability of 0.5. Lastly, Gaussian noise with a standard deviation of 0.05 is added to simulate real-world noise and encourage the model to learn correlations between different features. These transformations ensure the model is exposed to a wider range of data during training than the plain original dataset would allow. Figure 1 demonstrates the applied transformations.

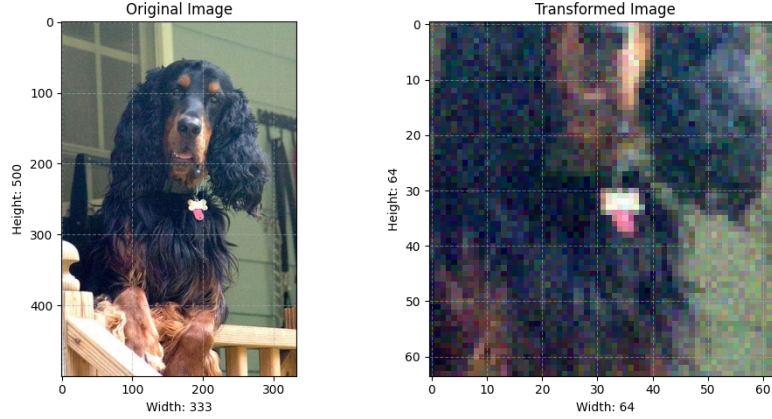


Figure 1: Comparison of an original image (left) and its transformed version (right) after augmentation (random cropping, horizontal flipping with 0.5 chance & addition of gaussian noise). In this case the image was not flipped.

4 Convolutional Autoencoder

4.1 Model Architecture

The full architecture is visualized in figure 2. The encoder of our CAE consists of three convolutional layers, each of them followed by a batch normalization layer and a ReLU activation function. We choose to downsample using convolutional layers with stride 2, as this is a learnable approach to downsampling, thus yielding potentially more optimal downsampling than with pooling layers. The encoder outputs a latent vector as the compressed version of the input. The size of this vector can be specified with a hyperparameter called *latent_dim*. We want the model to be small and to be able to visualize the learned latent space after training with respect to class labels, which is why we try to have a low latent space dimension. At first we set *latent_dim* to 256, but this gives the model too little capacity and it gets stuck training, being unable to learn compression and decompression. After increasing *latent_dim* to 512 the model has enough capacity to learn compression successfully.

The decoder receives a latent vector as input and constructs a $64 \times 64 \times 3$ image out of it by upsampling via transposed convolutional layers. This is again a learnable method for upsampling, providing greater possibility for optimal upsampling than unlearnable methods. The sigmoid activation function after the final layer scales the output pixel values to $[0,1]$, in order to match the original input of the encoder.

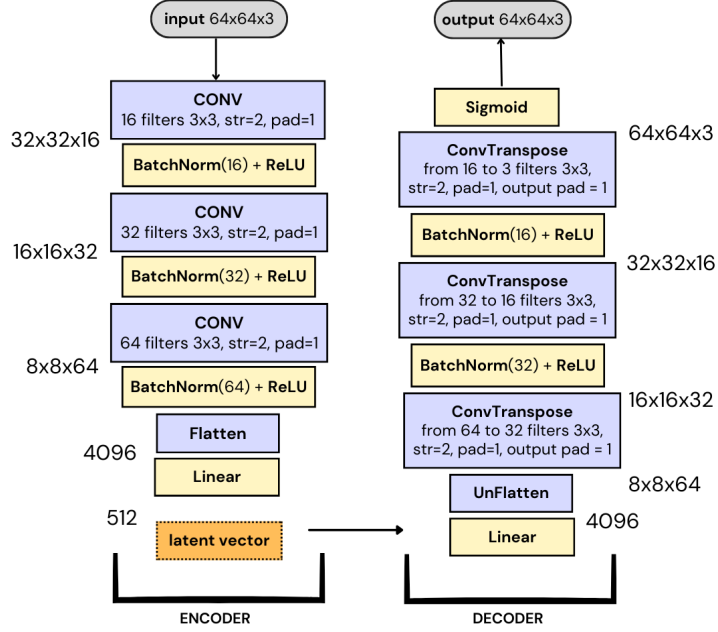


Figure 2: Illustration of the CAE Model Architecture. The output sizes of the respective layers are written on the side.

4.2 Model Training

The CAE’s layers are initialized using methods tailored to their specific activation functions and roles in the network. Convolutional and transposed convolutional layers were initialized with Kaiming (He) initialization to support ReLU activations. Linear (fully connected) layers also used Kaiming initialization, while the final transposed convolution layer in the decoder employed Xavier initialization due to the following sigmoid.

The model is trained with the Adam optimizer and the MSE loss. MSE fits our task, as we want the model to minimize the difference between inputs and reconstructed images during training. Using a learning rate of 1×10^{-3} proved to be too high, resulting in early stagnation of the loss, while a learning rate of 1×10^{-4} resulted in overly long training. We solve both problems by using a Cosine Annealing LR scheduler starting with a learning rate of 1×10^{-3} and decreasing it to 0 over the course of training. A batch size of 32 yields the most consistent results in our setup. The loss converged after 18 epochs and both training and validation loss curves can be seen in figure 3. These curves demonstrate how the model improves over time, with both losses typically decreasing as the model learns to reconstruct the input images more accurately. The validation loss curve consistently decreasing and not suddenly increasing shows that the model is clearly not overfitting.

We visually assess the quality of the image reconstructions produced by the model. A set of images from the training dataset is passed through the model, and the original images are compared to their corresponding reconstructions. The results can be seen in figure 4, where the first row shows the original images and the second row shows the reconstructed images generated by the model. By human oracle judgement the model seems to reconstruct images well, as the loss curves would suggest. However, a notable limitation of the reconstructed images is the presence of visible blurriness when compared to the originals. This blurriness likely arises from the lossy nature of the latent space representation, where fine-grained details are not preserved during encoding and decoding. As a result, while the overall structure is maintained, finer textures and details are lost during reconstruction. The generative power of the CAE is assessed in the results section.

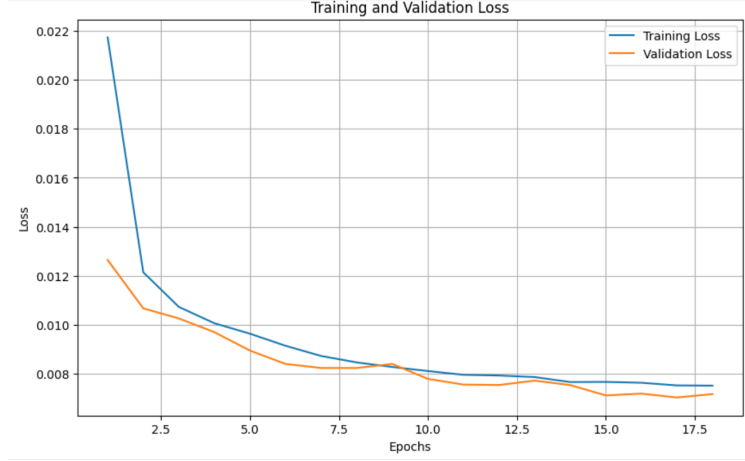


Figure 3: Training and Validation Loss Curves

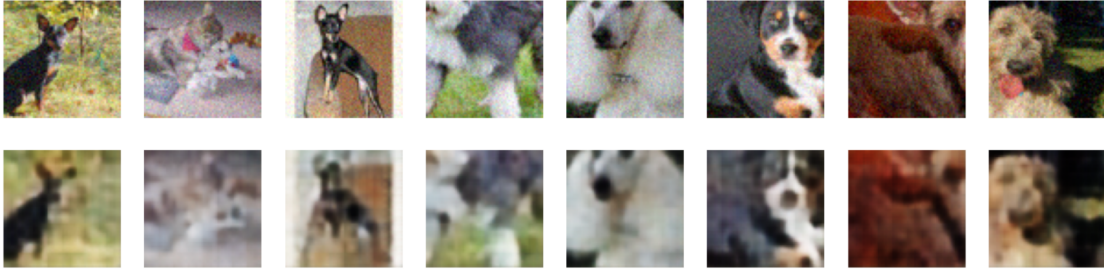


Figure 4: Original and Reconstructed Images

5 Variational Autoencoder

Building on the previous discussion of the Convolutional Autoencoder (CAE), the Variational Autoencoder (VAE) introduces a probabilistic approach to encoding and decoding data. Instead of learning a deterministic mapping, the VAE models a distribution over the latent variables, allowing for more flexible and diverse representations. This enables the generation of new, realistic samples, which is particularly valuable in tasks such as generating diverse and high-quality dog images.

5.1 Model Architecture

The Variational Autoencoder (VAE), like the Convolutional Autoencoder (CAE), is composed of two primary components: an encoder and a decoder. These components are interconnected through a probabilistic latent space.

The encoder maps the input image to a latent space characterized by a mean vector μ and a log-variance vector $\log \sigma^2$. These parameters define a Gaussian distribution from which the latent vector z is sampled. The encoder uses three convolutional layers to progressively downsample the spatial dimensions of the input image from $64 \times 64 \times 3$ (RGB) to $8 \times 8 \times 64$. Batch normalization and LeakyReLU activations are applied after each convolutional layer to improve stability and non-linearity. The output of the final convolutional layer is flattened and passed through two fully connected layers to compute μ and $\log \sigma^2$.

The decoder reconstructs the image from the latent vector z . It begins with a fully connected layer that expands the latent vector into the shape $8 \times 8 \times 64$. The decoder then applies three transposed convolutional layers to progressively upscale the spatial dimensions to $64 \times 64 \times 3$. Batch normalization and LeakyReLU activations are used in intermediate layers, while the final layer applies a sigmoid activation to ensure pixel values are within the range $[0, 1]$.

The model employs the reparameterization trick to enable backpropagation through the stochastic sampling process. The latent vector z is computed as $z = \mu + \epsilon \cdot \sigma$, $\epsilon \sim \mathcal{N}(0, I)$, where $\sigma = \exp(0.5 \cdot \log \sigma^2)$.

This step introduces stochasticity while maintaining differentiability. To illustrate this process, Figure 5 presents a graphical representation of how the probabilistic latent space is structured:

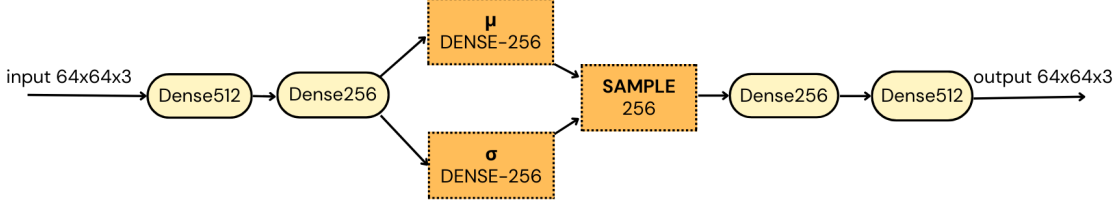


Figure 5: Graphical representation of the probabilistic latent space in the VAE. The encoder maps the input image to two vectors, μ and σ , which define the Gaussian distribution from which the latent vector $\mathbf{z}$ is sampled. This stochastic sampling introduces variability while retaining differentiability.

The loss we used for our model consists of two parts: a reconstruction loss computed as mean-square error, and a KL divergence loss. The KL divergence forces the model to learn a global continuous latent space (i.e. clusters of classes must be close to each other to allow interpolation between classes). Maintaining a good ratio of the reconstruction loss and the KL divergence is the key to generate good images. If the reconstruction loss is weighted higher, then the reconstructed images will be sharper, but the generated images will look like unstructured blob as the latent space is not smooth enough. If the KL divergence is weighted higher, then the generated images will have some structure resembling a dog, but the reconstruction power of the model will be worse, that can lead to lost of details or blurry images if the model is not deep enough to learn all of the features.

5.2 Model Training

The VAE was trained using a latent dimension of 256, which provides a compact representation of the input images. A batch size of 32 was selected to balance memory usage and training stability, and the learning rate was set to 10^{-4} , optimized using the Adam optimizer. The input images were resized to 64×64 RGB format for consistency. One of a major difference comparing with the traditional autoencoder is the introduction of the KL loss term. We introduce a new hyperparameter `kl_weight` so that we can overweight or underweight the KL loss when training. We found that `kl_weight =  $10^{-6}$` gives the best results. As shown in Figure 6, the loss function steadily decreased during training, demonstrating the effectiveness of the training procedure.

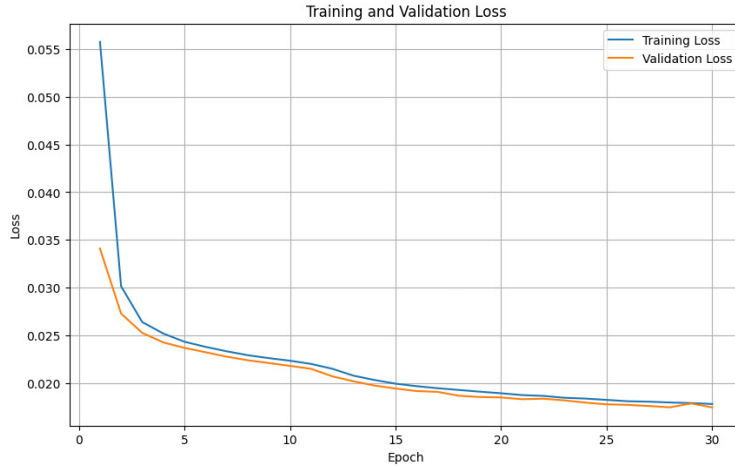


Figure 6: Training and Validation Loss Curves of VAE

We also find that using LeakyRelu instead of Relu does not help our model much, maybe because this dataset does not cause deadRelu. LeakyRelu makes our model train slower, but not by a huge margin, so we decided to keep using it just to be safe.

To improve the convergence and stability of the model, weight initialization was applied. Convolutional and transposed convolutional layers were initialized using Kaiming initialization, which is well-suited for LeakyReLU activations. Fully connected layers were also initialized using Kaiming initialization. For the second-to-last layer in the decoder,

Xavier initialization was applied to ensure proper scaling. The quality of the generated images was evaluated, and some of the reconstructed outputs are shown in Figure 7, where the structural patterns of the images are evident despite some blurriness.

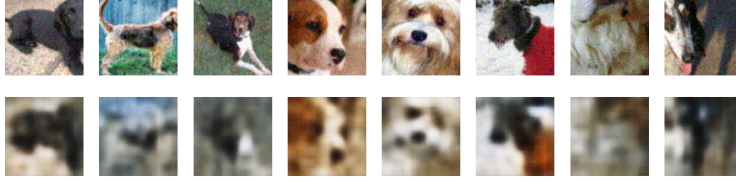


Figure 7: Original and Reconstructed Images

5.3 Deep VAE

Since the generated images are still not good enough, we think that it is because it is too simple to learn the features of the dataset, and the receptive field is too small. So we try to train a deeper VAE (with 6 layers instead of 3), but it still have some problems and does not perform so well.

6 Evaluation and Results

The evaluation of the models combines qualitative observations of the generated images and quantitative assessments using the Inception Score (IS) and Fréchet Inception Distance (FID). The IS (the higher, the better) is calculated by passing generated images through an Inception network pre-trained on ImageNet. It serves as a quick indicator of the quality and diversity of generated images. However, the IS does not compare generated images to the original dataset and, therefore, may not fully reflect realism. In contrast, the FID (the lower, the better) measures the distance between the distributions of generated and real images, computed in the feature space of the Inception network. FID provides a more robust evaluation by assessing how closely the generated images resemble the real dataset.

The images generated by the Convolutional Autoencoder (CAE), shown in Figure 8, exhibit highly abstract patterns with scattered pixels and lack discernible shapes or features.

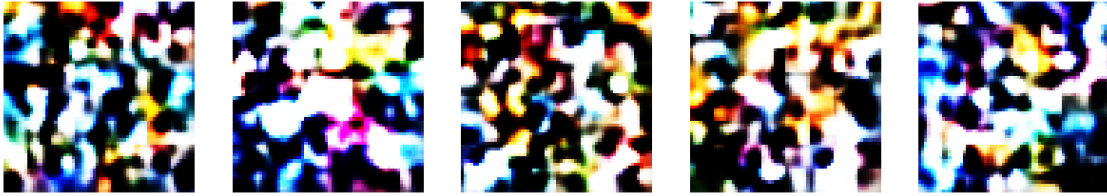


Figure 8: Generated images of the CAE when randomly sampling from the latent space.

This highlights that the CAE struggles to generate meaningful samples from the latent space, as expected, given that CAEs are not inherently designed for generative tasks. Quantitatively, the CAE achieved an IS of 4.9470 (std: 0.8091), reflecting moderate diversity and quality in reconstructed images. However, its high FID of 331.6248 underscores a substantial gap between the generated and real image distributions, emphasizing its limitations in generating realistic samples.

In contrast, the Variational Autoencoder (VAE) generated images with slightly more structure, as shown in Figure 9. While still blurry and lacking fine details, the outputs exhibit vague patterns and textures reminiscent of the original data, such as fur-like structures.

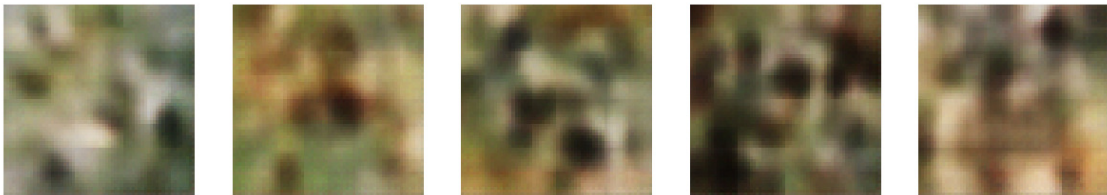


Figure 9: Generated images of the VAE when randomly sampling from the latent space.

The VAE’s probabilistic approach enables more coherent sampling from the latent space compared to the CAE. However, its IS of 1.5996 (std: 0.0355) and significantly higher FID of 938.6227 suggest limitations in both image quality and diversity, with the high FID indicating weaker alignment with real image distributions.

7 Discussion

The results show that moving from the CAE to the VAE has the expected effect of much better image generation quality. However the quality is still far from perfect, which is likely due to the small model capacity. This can be expected considering the complexity of the data.

A short-coming of our work is missing inspection of the latent space. We did inspect it however and none of the classes were grouped as one would expect (in the case of the CAE after training). This suggests that even though the model can reconstruct, it does not learn to do so based on classes. This could be further investigated, for example by limiting training to samples of only a handful of classes. This might then also allow for class-specific image generation later on.

References

- [1] Dor Bank, Noam Koenigstein and Raja Giryes. *Autoencoders*. 2021. arXiv: 2003.05991 [cs.LG]. URL: <https://arxiv.org/abs/2003.05991>.
- [2] Carl Doersch. *Tutorial on Variational Autoencoders*. 2021. arXiv: 1606.05908 [stat.ML]. URL: <https://arxiv.org/abs/1606.05908>.
- [3] Ashhadul Islam and Samir Brahim Belhaouari. ‘Fast and Efficient Image Generation Using Variational Autoencoders and K-Nearest Neighbor OverSampling Approach’. In: *IEEE Access* 11 (2023), pp. 28416–28426. DOI: 10.1109/ACCESS.2023.3259236.
- [4] Aditya Khosla et al. ‘Novel Dataset for Fine-Grained Image Categorization’. In: *First Workshop on Fine-Grained Visual Categorization, IEEE Conference on Computer Vision and Pattern Recognition*. Colorado Springs, CO, June 2011.
- [5] Diederik P. Kingma and Max Welling. ‘Auto-Encoding Variational Bayes’. In: *CoRR* abs/1312.6114 (2013). URL: <https://api.semanticscholar.org/CorpusID:216078090>.
- [6] Troy Luhman and Eric Luhman. *High Fidelity Image Synthesis With Deep VAEs In Latent Space*. 2023. arXiv: 2303.13714 [cs.CV]. URL: <https://arxiv.org/abs/2303.13714>.
- [7] Gaurav Raut and Apoorv Singh. *Generative AI in Vision: A Survey on Models, Metrics and Applications*. 2024. arXiv: 2402.16369 [cs.CV]. URL: <https://arxiv.org/abs/2402.16369>.
- [8] D. E. Rumelhart, G. E. Hinton and R. J. Williams. ‘Learning internal representations by error propagation’. In: *Parallel Distributed Processing: Explorations in the Microstructure of Cognition, Vol. 1: Foundations*. Cambridge, MA, USA: MIT Press, 1986, pp. 318–362. ISBN: 026268053X.
- [9] Mirza Tanzim Sami and Iftekharul Mobin. ‘A Comparative Study on Variational Autoencoders and Generative Adversarial Networks’. In: Apr. 2019. DOI: 10.1109/ICAIIIT.2019.8834544.
- [10] Axel Sauer et al. *Projected GANs Converge Faster*. 2021. arXiv: 2111.01007 [cs.CV]. URL: <https://arxiv.org/abs/2111.01007>.
- [11] Sanchayan Vivekananthan. *Comparative Analysis of Generative Models: Enhancing Image Synthesis with VAEs, GANs, and Stable Diffusion*. 2024. arXiv: 2408.08751 [cs.CV]. URL: <https://arxiv.org/abs/2408.08751>.
- [12] Yingche Xu et al. ‘Image Reconstruction by VAE’. In: *Cambridge Explorations in Arts and Sciences* 1 (Dec. 2023). DOI: 10.61603/ceas.v1i2.20.